

Final Year Project Proposal: Gesturelink

Title:

Gesturelink - A Real-Time Sign-Assisted Video Conferencing System

Student: Francis Waithaka Njoroge

Admission: BTCEs/2022/51289

Department: Bachelor of Technology in Computer and Electronics Systems

TABLE OF CONTENTS

Final Year Project Proposal: Gesturelink.....	1
Title:.....	1
TABLE OF CONTENTS.....	2
1. Project Overview.....	3
2. Problem Statement.....	3
3. Objectives.....	3
Main Objective.....	3
Specific Objectives.....	3
4. System Architecture.....	4
a. Client Side.....	5
b. Web Interface.....	6
c. Server Side.....	7
5. Methodology.....	7
6. Expected Outcomes.....	8
7. Tools and Technologies.....	8

1. Project Overview

Gesturelink is an **AI-driven, real-time video conferencing system** that integrates **sign language interpretation and speech synthesis** to promote inclusive digital communication. The system bridges interaction between hearing, mute, and visually impaired individuals by translating **sign language to speech** and **speech to sign language** dynamically during live video sessions.

Gesturelink leverages **computer vision, machine learning, and natural language processing (NLP)** for multimodal translation, ensuring accessibility and understanding across diverse users in both personal and professional virtual communication settings.

2. Problem Statement

Current video conferencing platforms such as **Zoom, Google Meet, Microsoft Teams, and Signal** lack accessibility mechanisms that fully accommodate users with communication disabilities.

Deaf users cannot follow spoken discussions, and mute users struggle to participate effectively, while visually impaired individuals face challenges in interpreting visual gestures.

Gesturelink aims to eliminate these barriers by introducing a **bi-directional translation framework** — translating speech to sign and sign to speech — to enable **seamless, inclusive, and equitable communication**.

3. Objectives

Main Objective

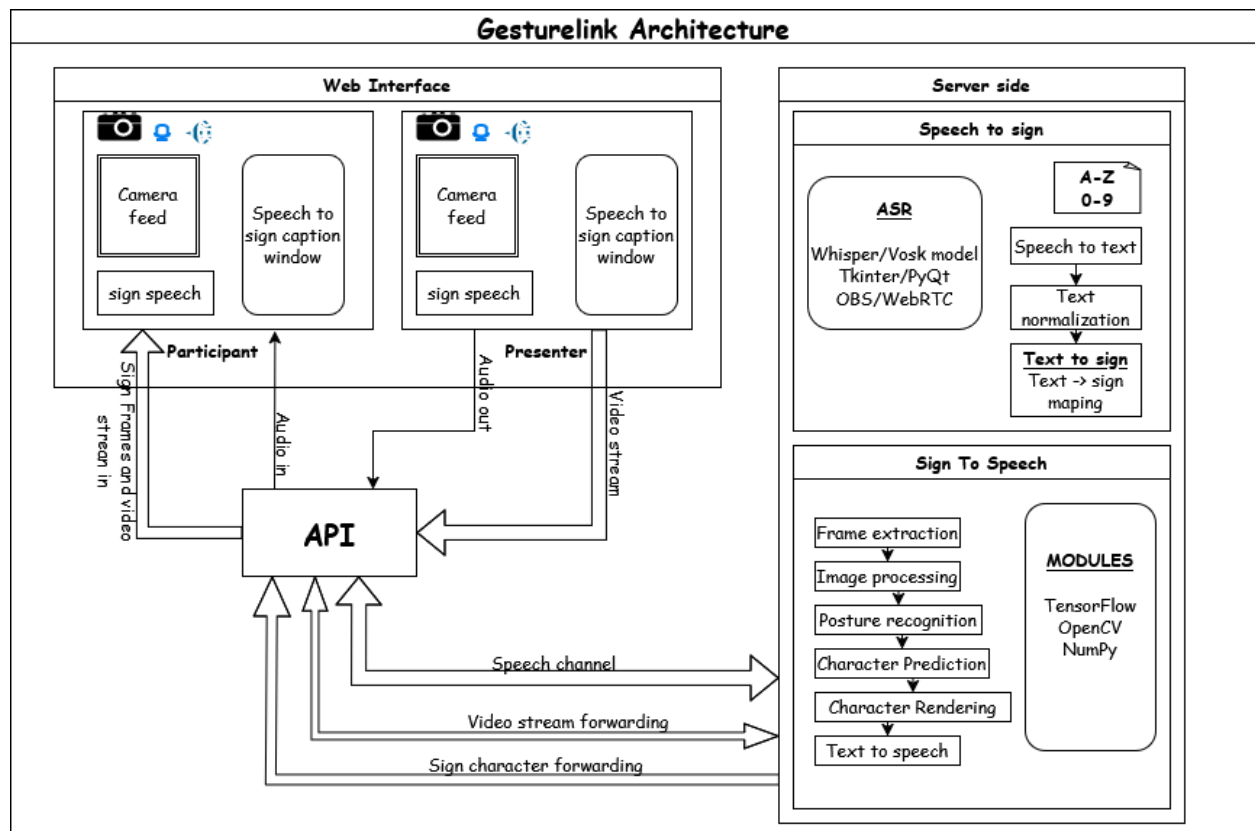
To design and implement a real-time, web-based video conferencing platform with integrated **sign-to-speech** and **speech-to-sign** translation.

Specific Objectives

1. Develop a **web interface** for dual-pane live conferencing with sign and speech captions.
2. Implement **real-time gesture recognition** using OpenCV and TensorFlow.
3. Integrate **Automatic Speech Recognition (ASR)** using Whisper or Vosk for speech-to-text conversion.
4. Implement **text-to-sign rendering** through a mapping engine (A-Z, 0-9, and common words).
5. Integrate **text-to-speech** for visually impaired users.
6. Ensure **low-latency transmission** for near real-time interaction.
7. Enable **OBS/WebRTC integration** for compatibility with existing conferencing platforms (Zoom, Meet, Signal).

4. System Architecture

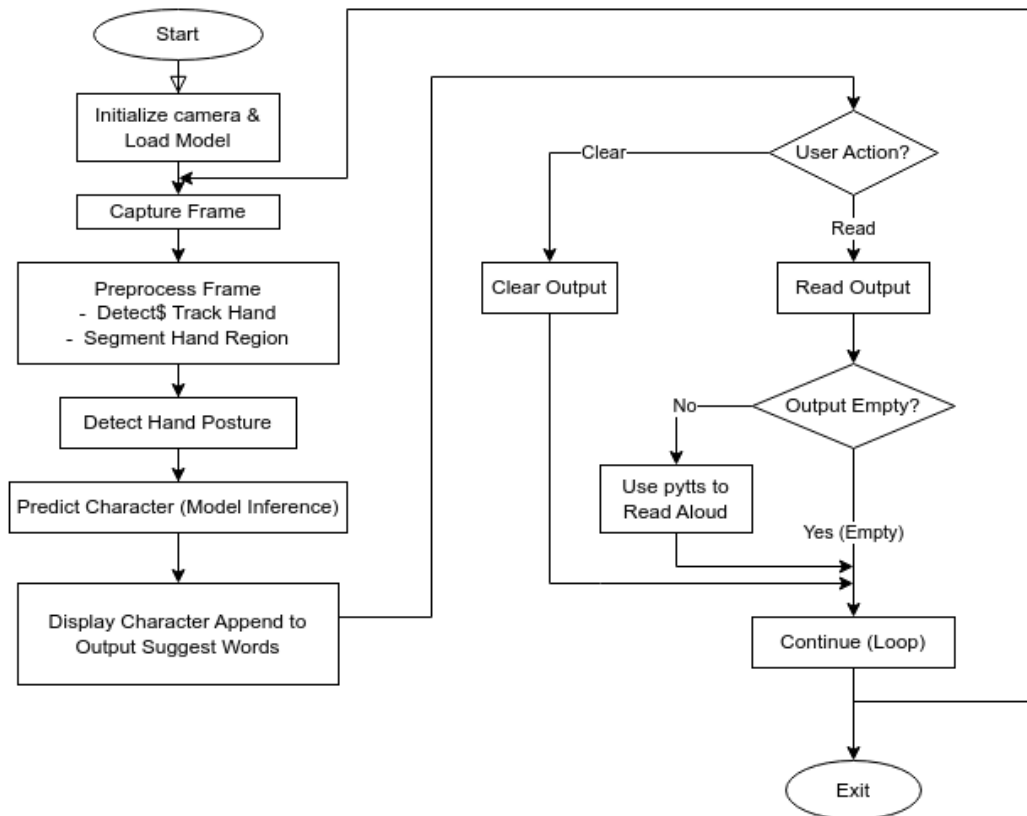
The **Gesturelink** architecture (shown in the attached diagram) is composed of three major subsystems:



a. Client Side

- **Sign Interpreter Module:**

Captures hand gestures through the user's webcam. Frames are preprocessed using OpenCV and passed through a trained TensorFlow model to infer corresponding sign characters or words.



- **Speech Input Module:**

Captures and transcribes user speech using Whisper/Vosk, producing text captions for conversion into sign representations.

-

b. Web Interface

- Built using **HTML/CSS/JavaScript**, the interface provides:
 - Dual video panes for participant and presenter.

- Real-time **speech-to-sign** and **sign-to-speech** caption windows.
- **Auto-scroll controller** to synchronize display timing with speech rate or gesture frequency.

c. Server Side

- Powered by **Flask/FastAPI**, the server exposes RESTful endpoints for:
 - **/upload-frame** - Capture and preprocess gesture frames.
 - **/infer-posture** - Predict gesture labels.
 - **/speech-to-text** - Perform ASR processing.
 - **/text-to-sign** - Map text to sign visuals.
 - **/speak-output** - Convert recognized text to speech.

Modules Used:

- **Speech to Sign:** Whisper/Vosk ASR → Text Normalization → Text-to-Sign Mapping.
- **Sign to Speech:** Frame Extraction → Image Processing → Posture Recognition → Character Prediction → Text-to-Speech Rendering.

5. Methodology

Phase	Description
Data Collection	Use standard sign gesture datasets (A-Z, 0-9) and locally recorded sign video samples.
Model Training	Train gesture recognition models using TensorFlow and OpenCV.
ASR Integration	Configure Whisper/Vosk for accurate English speech transcription.
GUI Development	Design a Tkinter-inspired web interface using HTML, CSS, and JavaScript.
Integration	Connect video and audio pipelines through Flask API and WebRTC/OBS.
Testing	Evaluate recognition accuracy, latency, and user experience with real participants.

6. Expected Outcomes

1. A **fully functional prototype** of Gesturelink enabling real-time speech-to-sign and sign-to-speech translation.
2. **Cross-platform compatibility** with video conferencing tools like Zoom and Signal via virtual camera feed.
3. Enhanced **accessibility and inclusion** for users with hearing or speech impairments.
4. A foundation for future research in **assistive AI communication technologies**.

7. Tools and Technologies

Category	Tools / Frameworks
Programming	Python, JavaScript
ML Frameworks	TensorFlow, OpenCV, NumPy
ASR Models	Whisper, Vosk
Backend	Flask / FastAPI
Frontend	HTML, CSS, JavaScript
Video Integration	OBS, WebRTC
Development Tools	Jupyter, VSCode, Postman